USER MANUAL

Q_{10} Calculator for Circadian Period

Browser, desktop (Python) and Google Colab versions

Estimating the temperature coefficient Q_{10} and the Arrhenius activation energy of a circadian clock from period measurements at several temperatures.

$$\tau(T) = \tau_b \cdot Q_{10}^{\frac{T_b - T}{10}}$$

Covers: index.html (browser), q10_calculator.py (desktop), circadian_period_q10.ipynb (Google Colab)

Repository: github.com/dxda6216/q10

Manual date: October 2026

Contents

1 Introduction 3

1.1 What the calculator does 3

1.2 Three versions 3

1.3 Which version should I use? 3

1.4 Conventions in this manual 4

2 Background 5

2.1 The temperature coefficient Q_{10} 5

2.2 Temperature compensation of circadian clocks 5

2.3 The model that is fitted 5

2.4 The base temperature 6

2.5 The Arrhenius plot and its relation to Q_{10} 6

2.6 The Arrhenius regression line 7

3 Calculation method 8

3.1 Objective 8

3.2 Starting values 8

3.3 Optimisation 8

3.4 Standard errors 8

3.5 Goodness of fit 8

3.6 Fitted curve 9

3.7 Arrhenius regression 9

3.8 Agreement between the versions 9

4 Preparing your data 10

4.1 Requirements 10

4.2 Data formats 10

4.3 Replicates and averages 10

4.4 Choosing a temperature range 11

4.5 Choosing a base temperature 11

5 Browser version 12

5.1 Getting and opening the file 12

5.2 Screen layout 12

5.3 Step by step 13

5.4 The plots 14

5.5 The report 15

5.6 Saving 15

5.7 The “How it works” window 15

5.8 Phones, tablets and dark mode 16

6 Python version 17

6.1 Requirements 17

6.2 Installation 17

6.3 Anaconda and Miniconda 17
6.4 Starting the program 18
6.5 The window 18
6.6 Step by step 19
6.7 Saving 19

7 Google Colab version 20

7.1 Opening the notebook 20
7.2 Form fields 20
7.3 Running and output 21
7.4 Saving 21
7.5 Using the notebook in Jupyter 21

8 Output reference 22

8.1 Results text 22
8.2 Plots 22
8.3 Tab-delimited data 23

9 Worked example 24

9.1 All points, base temperature 37 °C 24
9.2 Fit range 28–40 °C 24
9.3 Reading the example 25

10 Interpreting the results 26

11 Error messages and troubleshooting 27

11.1 Error messages 27
11.2 Troubleshooting 27

12 Privacy and data handling 29

13 Differences between the versions 30

Appendix A Formula summary 31

Appendix B Glossary 32

Appendix C Third-party components 33

1 Introduction

1.1 What the calculator does

The Q_{10} calculator estimates how strongly the period of a circadian rhythm depends on temperature. You give it period measurements made at several temperatures; it fits the standard Q_{10} model to the data and reports:

- the temperature coefficient $\mathbf{Q_{10}}$ with its standard error;
- the **estimated period** $\boldsymbol{\tau_b}$ at a base temperature of your choice, with its standard error;
- the coefficient of determination $\mathbf{R^2}$ of the fit;
- an **Arrhenius regression** of ln(frequency) on 1000/T, giving the **activation energy** $\mathbf{E_a}$, the slope and intercept with standard errors, and R^2.

The results are shown as three plots (period, frequency and Arrhenius) and as a text report that you can save or paste into a spreadsheet.

1.2 Three versions

The same calculator is available in three versions. They use the same model, the same fitting method and the same inputs, and produce the same numbers (see Section 3.8).

	Browser (HTML)	Desktop (Python)	Google Colab
File	`index.html`	`q10_calculator.py`	`circadian_period_q10.ipynb`
Runs on	Your computer, in a web browser	Your computer	Google's servers
Installation	None	Python with numpy, SciPy and matplotlib	None (a Google account is needed)
Internet	Not needed once the file is downloaded	Not needed	Required
Data leaves your computer	No	No	Yes: processed by Google Colab
Saving	PNG, SVG figure; TXT report	PNG, PDF, SVG figure; TXT results	Save image; copy text

1.3 Which version should I use?

- **Browser version** — the easiest choice for most people. Nothing to install, works on any computer, phone or tablet with a modern browser, and keeps your data on your own device. Good for unpublished data and for sharing the tool with colleagues.
- **Python version** — for people who already use Python, need a PDF figure, want to zoom into the plots, or want to read and adapt the code.
- **Colab version** — when you want to run the calculation without downloading anything, or prefer a notebook. Note that your data is processed on Google's servers.

1.4 Conventions in this manual

Names of buttons, fields and tabs are printed like **Calculate**. Keys are shown as Ctrl + Enter; on a Mac use ⌘ instead of Ctrl in the browser version. Temperatures are in degrees Celsius (°C) unless stated otherwise; T in the Arrhenius equations is the absolute temperature in kelvin ($K = °C + 273.15$). Periods are in hours, and frequencies in cycles per day (day^{-1}). All numerical examples use the example data built into the calculator (Section 9).

2 Background

2.1 The temperature coefficient Q_{10}

Q_{10} is the factor by which the rate of a process changes when the temperature rises by 10 °C. If the rate is R_1 at temperature T_1 and R_2 at T_2,

$$Q_{10} = \left(\frac{R_2}{R_1}\right)^{\frac{10}{T_2 - T_1}}$$

For example, $Q_{10} = 2$ means the rate doubles for every 10 °C of warming. Most enzyme-catalysed reactions have Q_{10} of about 2 to 3.

2.2 Temperature compensation of circadian clocks

For a circadian clock, the natural "rate" is its frequency: the number of cycles per unit time, which is the reciprocal of the period. A defining property of circadian clocks is **temperature compensation**: although the underlying biochemistry speeds up with temperature, the period stays nearly constant over the physiological range. Q_{10} of a circadian period is therefore typically close to 1, roughly between 0.8 and 1.2.

Value	Meaning
$Q_{10} > 1$	The period shortens as temperature rises: the clock runs faster (under-compensation).
$Q_{10} = 1$	The period does not change with temperature: perfect compensation.
$Q_{10} < 1$	The period lengthens as temperature rises: the clock runs slower (over-compensation).

2.3 The model that is fitted

Applying the definition of Q_{10} to frequency (rate = 1/period) and writing τ_b for the period at a reference or *base* temperature T_b gives the model fitted by the calculator:

$$\tau(T) = \tau_b \cdot Q_{10}^{\frac{T_b - T}{10}} \tag{1}$$

The frequency in cycles per day is

$$f(T) = \frac{24}{\tau(T)}$$

and taking logarithms turns the model into a straight line in T with slope $\ln(Q_{10})/10$:

$$\ln f(T) = \ln \frac{24}{\tau_b} + \frac{T - T_b}{10} \ln Q_{10} \tag{2}$$

Equation (2) is why a plot of ln(frequency) against temperature would be a straight line if the model held exactly. The calculator, however, fits equation (1) directly to the periods (see Section 3.1).

2.4 The base temperature

The base temperature T_b does not change the model; it only decides *at which temperature* the fitted period τ_b is reported. Changing T_b is a reparameterisation, so:

- Q_{10}, its standard error, R^2 and the fitted curve are identical for every choice of base temperature;
- only τ_b and its standard error change.

With the example data, for instance, τ_b is 23.662 ± 0.221 h at 37 °C and 28.840 ± 0.232 h at 30 °C, while Q_{10} = 1.327 ± 0.019 in both cases.

> A base temperature far outside your data — absolute zero (−273.15 °C) is offered as an option — makes τ_b a long extrapolation of the model, with a correspondingly large standard error. Use such values only if you specifically need them.

2.5 The Arrhenius plot and its relation to Q_{10}

The Arrhenius equation describes the temperature dependence of a rate constant k through an activation energy E_a:

$$k = A \cdot \exp\left(-\frac{E_a}{RT}\right)$$

where A is a pre-exponential factor, R = 8.314 J mol^{-1} K^{-1} is the gas constant and T is the absolute temperature. Taking logarithms gives a straight line in 1/T with slope $-E_a/R$:

$$\ln k = \ln A - \frac{E_a}{R} \cdot \frac{1}{T}$$

The calculator's Arrhenius plot shows ln(frequency) against 1000/T, with T in kelvin. Points that fall on a straight line in this plot follow the Arrhenius equation; the steeper the line, the larger E_a.

How E_a and Q_{10} are related

Both descriptions predict the frequency ratio between two temperatures T_1 and T_2 (in kelvin). From equation (2) and from the Arrhenius equation respectively,

$$\ln\frac{f_2}{f_1} = \frac{T_2 - T_1}{10}\ln Q_{10} \quad \text{and} \quad \ln\frac{f_2}{f_1} = \frac{E_a}{R}\left(\frac{1}{T_1} - \frac{1}{T_2}\right) \tag{3}$$

Setting the two expressions equal, and noting that $1/T_1 - 1/T_2 = (T_2 - T_1)/(T_1T_2)$, gives

$$E_a = \frac{RT_1T_2\ln Q_{10}}{10} \quad \text{equivalently} \quad Q_{10} = \exp\left(\frac{10E_a}{RT_1T_2}\right) \tag{4}$$

This conversion is exact for a single pair of temperatures. Over a range of temperatures it is only approximate, because the two models have different shapes: the Q_{10} model has a constant slope in T, the Arrhenius model a constant slope in 1/T. Their local slopes agree when

$$\frac{d\ln f}{dT} = \frac{\ln Q_{10}}{10} \quad (Q_{10}\text{model}) \qquad \frac{d\ln f}{dT} = \frac{E_a}{RT^2} \quad (\text{Arrhenius})$$

so the same E_a corresponds to a slightly different Q_{10} at each temperature. Across a typical range of 15–20 °C the difference is small.

2.6 The Arrhenius regression line

In addition to the fitted Q_{10} curve, the calculator can draw the ordinary least-squares straight line of ln(frequency) on 1000/T in the Arrhenius plot. It is fitted to the same data points used for Q_{10}:

$$\ln f = b_0 + b_1 \cdot \frac{1000}{T} \qquad E_a = -1000 \cdot R \cdot b_1 \tag{5}$$

This line is shown by default (solid grey) and can be switched off. Its E_a, slope b_1, intercept b_0, standard errors and R^2 are always included in the results.

Why the two lines are close but not identical

1. **Different model forms.** The Q_{10} model is a straight line in T (equation 2); the Arrhenius model is a straight line in 1/T. Plotted against 1000/T, the Q_{10} curve is therefore slightly bent.
2. **Different residuals.** The Q_{10} fit minimises the squared errors of the *period*; the Arrhenius regression minimises the squared errors of *ln(frequency)*. The two criteria weight the data points slightly differently.

Within the range of the data the two lines nearly coincide — for the example data their largest difference is about 0.005 in ln(frequency), roughly 0.5 % in frequency and about one sixth of the scatter of the data about the line. Outside the data range, where both are extrapolations, they separate slightly. The E_a from the regression and the E_a converted from Q_{10} with equation (4) are also very close: 21.86 and 22.0 kJ/mol for the example data.

3 Calculation method

3.1 Objective

The parameters τ_b and Q_{10} are estimated by non-linear least squares on the periods themselves (not on their logarithms), minimising the sum of squared residuals

$$\mathrm{SSE} = \sum\nolimits_i \left(\tau_i - \widehat{\tau}\left(T_i \right) \right)^2$$

where τ_i is the measured period at temperature T_i and $\hat{\tau}(T_i)$ the period predicted by equation (1). All data points have equal weight. Only the points selected for the fit (all points, or those inside the chosen temperature range) are used.

3.2 Starting values

Non-linear fitting needs starting values. The calculator obtains them from a straight-line regression of ln(period) on temperature: by equation (2) its slope s and intercept c correspond to $\ln Q_{10} = -10s$ and $\ln \tau_b = c - (T_b/10)\cdot \ln Q_{10}$. These values are already close to the final solution, which makes the fit fast and reliable. If this step fails, the calculator starts from $\tau_b = 24$ h and $Q_{10} = 1$.

3.3 Optimisation

The Python and Colab versions use SciPy's `curve_fit` (for problems without bounds it uses the Levenberg–Marquardt algorithm of MINPACK). The browser version contains its own implementation of the Levenberg–Marquardt algorithm in JavaScript, written for this calculator. At each iteration it solves

$$\left(J^T J + \lambda \cdot \mathrm{diag}\left(J^T J \right) \right) \delta = J^T r$$

for the parameter step δ, where J is the Jacobian of the model with respect to the parameters and r the vector of residuals. After a step that lowers SSE, the damping factor λ is reduced (moving towards the Gauss–Newton method); otherwise it is increased (moving towards steepest descent) and the step is retried. The browser version iterates on $\ln \tau_b$ and $\ln Q_{10}$, which keeps both parameters positive; this does not change the objective, so it converges to the same minimum.

3.4 Standard errors

Standard errors are taken from the covariance matrix of the parameters,

$$\mathrm{Cov} = s^2 \left(J^T J \right)^{-1}, \qquad s^2 = \frac{\mathrm{SSE}}{n-2}$$

where J is evaluated at the solution with respect to τ_b and Q_{10}, and n is the number of points used in the fit. The ± values in the results are the square roots of the diagonal elements. This is the same definition that `curve_fit` uses by default (`absolute_sigma=False`): the errors are scaled by the observed scatter of the data about the fitted curve.

3.5 Goodness of fit

$$R^2 = 1 - \frac{\mathrm{SSE}}{\mathrm{SST}}, \qquad \mathrm{SST} = \sum \left(\tau_i - \bar{\tau} \right)^2$$

$\bar{\tau}$ is the mean of the periods used in the fit. R^2 is the fraction of the variation in period that the model explains; see Section 10 for how to read it.

3.6 Fitted curve

The fitted curve is drawn with 200 evenly spaced points. Its temperature range covers all the data (including points outside a chosen fit range) and extends on each side by one third (33.3 %) of the data's temperature span; the end points are truncated to whole degrees. For the example data (25.1–41.1 °C) the curve runs from 19 to 47 °C. The same span is used for the Arrhenius regression line.

3.7 Arrhenius regression

For each data point used in the fit,

$$x_i = \frac{1000}{T_i + 273.15}, \qquad y_i = \ln \frac{24}{\tau_i}$$

and the regression line $y = b_0 + b_1 x$ is found by ordinary least squares:

$$b_1 = \frac{S_{xy}}{S_{xx}}, \qquad b_0 = \bar{y} - b_1 \bar{x}$$

with $S_{xx} = \Sigma(x_i - \bar{x})^2$ and $S_{xy} = \Sigma(x_i - \bar{x})(y_i - \bar{y})$. With $s^2 = \mathrm{SSE}/(n - 2)$ from the regression residuals, the standard errors are

$$\mathrm{SE}\,(\,b_1\,) = \sqrt{\frac{s^2}{S_{xx}}}, \qquad \mathrm{SE}\,(\,b_0\,) = \sqrt{s^2 \left(\frac{1}{n} + \frac{\bar{x}^2}{S_{xx}} \right)}$$

and the activation energy follows from the slope:

$$E_a = -1000 R b_1, \qquad \mathrm{SE}\,(\,E_a\,) = 1000 R \cdot \mathrm{SE}\,(\,b_1\,)$$

using $R = 8.314462618$ J mol^{-1} K^{-1}. E_a is reported in kJ/mol.

3.8 Agreement between the versions

The three versions were checked against each other with the example data, with several base temperatures (30 °C, 37 °C, Median and absolute zero) and with a restricted range (28–40 °C). τ_b, Q_{10}, their standard errors, R^2 and all Arrhenius regression values agree to every printed digit. The Arrhenius regression values also agree with SciPy's `linregress`.

4 Preparing your data

4.1 Requirements

- Temperatures in **°C** and periods in **hours**.
- Exactly **one period for each temperature**.
- At least **3 data points** in the fit (two parameters are estimated, and standard errors need at least one degree of freedom). More points give more reliable standard errors.
- All periods must be **positive**.
- The same temperature may appear more than once (for example, for replicate measurements).

4.2 Data formats

Browser and Python versions: the data box

Type or paste one temperature–period pair per line. The two values can be separated by commas, spaces, tabs or semicolons, and these can be mixed. Blank lines and lines starting with `#` are ignored, and one header line at the top is skipped automatically, so data copied from a spreadsheet usually works as it is:

```
Temperature, Period
# Strain A, constant darkness
25.1, 33.6
26.9  32.8
27.3    30.4
```

Browser and Python versions: CSV files

Load CSV… reads a comma-, semicolon-, tab- or space-separated text file (.csv, .tsv or .txt). Temperature must be in the first column and period in the second; further columns are ignored. Rows whose first two cells are not both numbers (such as headers) are skipped. The rows read are written into the data box, so you can check and edit them before calculating.

Colab version: two lists

Enter the temperatures and the periods as two comma-separated lists in the same order:

```
Temperatures = 25.1, 26.9, 27.3, 29.2
Periods      = 33.6, 32.8, 30.4, 29.3
```

4.3 Replicates and averages

Every pair you enter counts as one independent observation with equal weight. You can enter either

- **individual measurements** — one line per animal, culture or recording, repeating the temperature as needed; or
- **mean periods** — one line per temperature.

Both are valid, but they answer slightly different questions and give different standard errors: with individual measurements, the standard errors reflect the scatter between replicates; with means, they reflect only the scatter of the means about the curve and are based on fewer points. State which you

used when you report results. The calculator does not apply weights, so means based on very different numbers of replicates are treated equally.

4.4 Choosing a temperature range

Circadian clocks are often well compensated within a physiological range and less so near its limits, where the period may change sharply or the rhythm may become unstable. The temperature-range option lets you fit Q_{10} to a chosen part of the data while still plotting all points. Points outside the range are shown in light green and are not used for any calculation, including the Arrhenius regression and the Minimum/Maximum/Average/Median base temperatures.

The choice can make a real difference: with the example data, Q_{10} is 1.327 ± 0.019 using all 18 points but 1.261 ± 0.026 using the 13 points between 28 and 40 °C. Decide on the range on biological grounds, ideally before looking at the result, and always report it.

4.5 Choosing a base temperature

Choose the temperature at which a period estimate is most useful — often a standard culture or body temperature (for example 25 °C for many insects and fungi, or 37 °C for mammalian cells), or a value inside your data such as the median. Because the base temperature does not affect Q_{10} (Section 2.4), you can change it freely to read off the fitted period at different temperatures.

Option	Value used
−273.15 °C, 0, 4, 25, 30, 37, 100 °C	That fixed temperature.
Minimum, Maximum, Average, Median	Computed from the temperatures of the points used in the fit.
Custom value	Any temperature you enter (in Colab, set with the **Base_temperature** slider, 0–100 °C).

5 Browser version

5.1 Getting and opening the file

1. Open github.com/dxda6216/q10/blob/main/index.html.
2. Click **Download raw file** (the download icon above the file contents) and save `index.html`.
3. Double-click the saved file, or open it from your browser's **File → Open** menu.

No installation, server or account is needed, and the file can be copied to other computers or sent to colleagues. Any current version of Chrome, Edge, Firefox or Safari works.

5.2 Screen layout

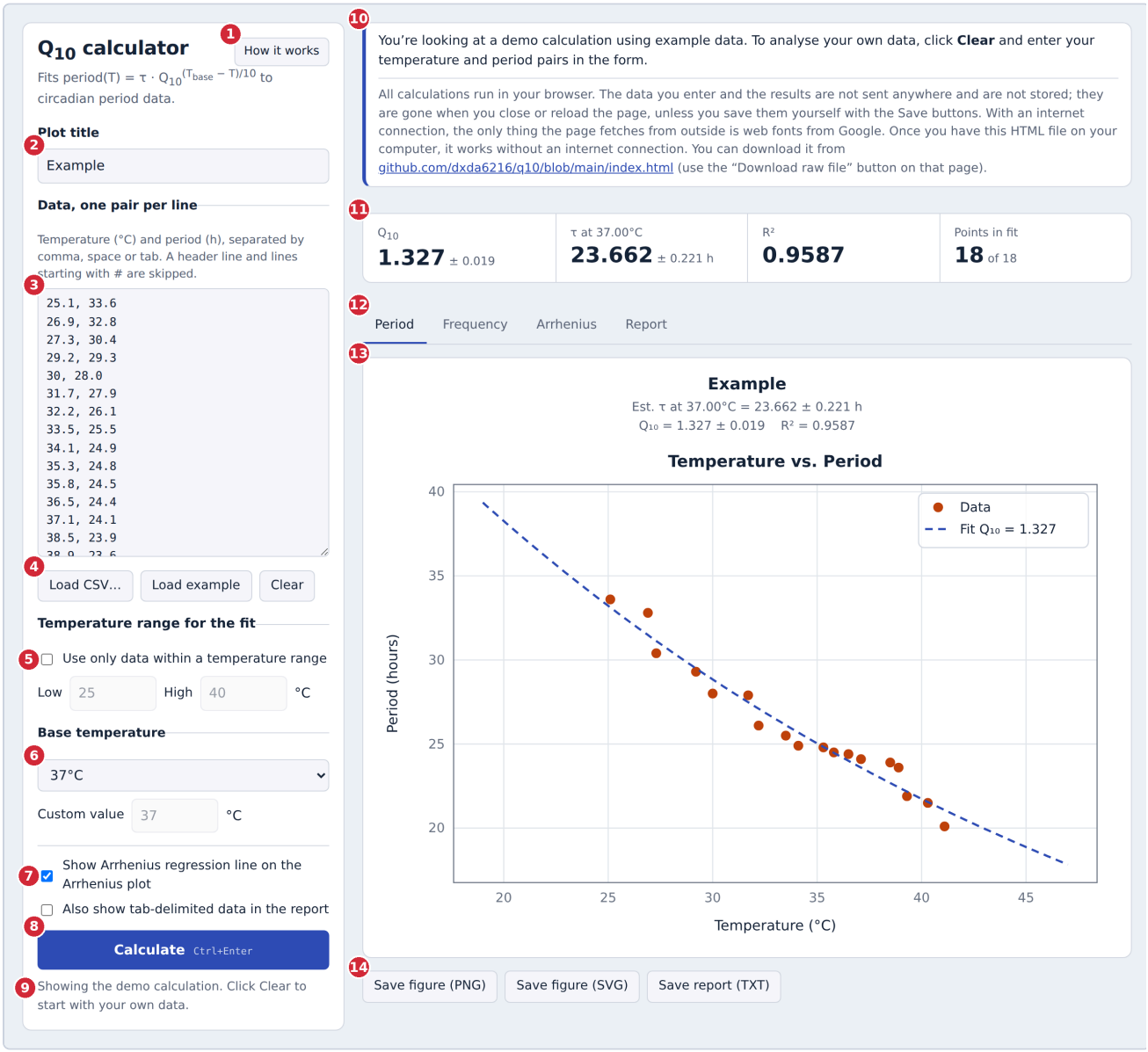


Figure 1. The browser version as it opens, showing the demo calculation. Numbers refer to the table below.

	Element	Purpose
1	**How it works**	Opens a window explaining the theory, the method, how to use the calculator and how your data is handled.
2	**Plot title**	Optional title shown above the plots and in saved figures; also used for saved file names.
3	**Data box**	Temperature–period pairs, one per line (Section 4.2).
4	**Load CSV… / Load example / Clear**	Read data from a file, reload the example data, or clear the data, the title and all results.
5	**Temperature range**	Tick to fit only points with Low ≤ T ≤ High; then set Low and High.
6	**Base temperature**	Temperature at which τ_b is reported; Custom value is enabled when "Custom value" is selected.
7	**Options**	Show the Arrhenius regression line; include tab-delimited data in the report.
8	**Calculate**	Runs the calculation (also Ctrl+Enter / ⌘+Enter).
9	**Status line**	Shows a short summary of the result, or the reason a calculation failed.
10	**Notice**	Shown on opening: explains the demo and how your data is handled. Hidden by Clear, by editing the data or by loading a CSV.
11	**Summary**	Q_{10}, τ_b (each ± standard error), R^2 and the number of points used in the fit.
12	**Tabs**	Switch between the Period, Frequency and Arrhenius plots and the text report.
13	**View**	The selected plot or report.
14	**Save buttons**	Save all three plots as one PNG or SVG image, or the report as a text file.

5.3 Step by step

1. **Look at the demo.** The page opens with the example data already calculated, so you can see what the results look like.
2. **Clear the form.** Click **Clear**. The notice, the example data and the results disappear.
3. **Enter your data.** Type or paste pairs into the data box, or use **Load CSV…**. Optionally enter a plot title.
4. **Set the options.** If needed, tick **Use only data within a temperature range** and set Low and High; choose the base temperature.
5. **Calculate.** Click **Calculate** or press Ctrl+Enter. The summary row and the plot update. If something is wrong with the input, the status line explains what (Section 11).
6. **Review.** Switch between the tabs. Changing the title or the Arrhenius-line checkbox updates the plots immediately; other changes need **Calculate** again.
7. **Save.** Use the save buttons (Section 5.6).

5.4 The plots

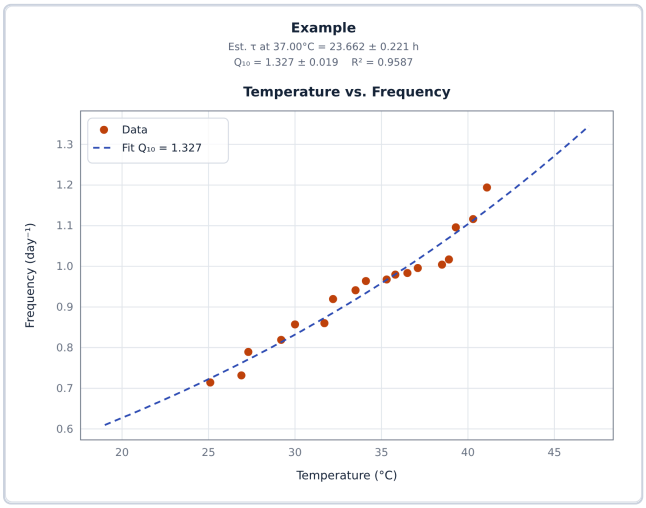


Figure 2. Frequency tab: frequency (day^{-1}) against temperature with the fitted Q_{10} curve.

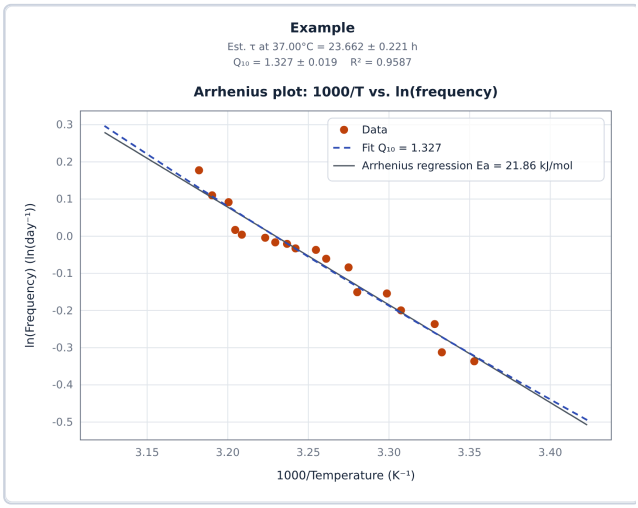


Figure 3. Arrhenius tab: ln(frequency) against 1000/T, with the fitted Q_{10} curve (dashed) and the Arrhenius regression line (solid grey).

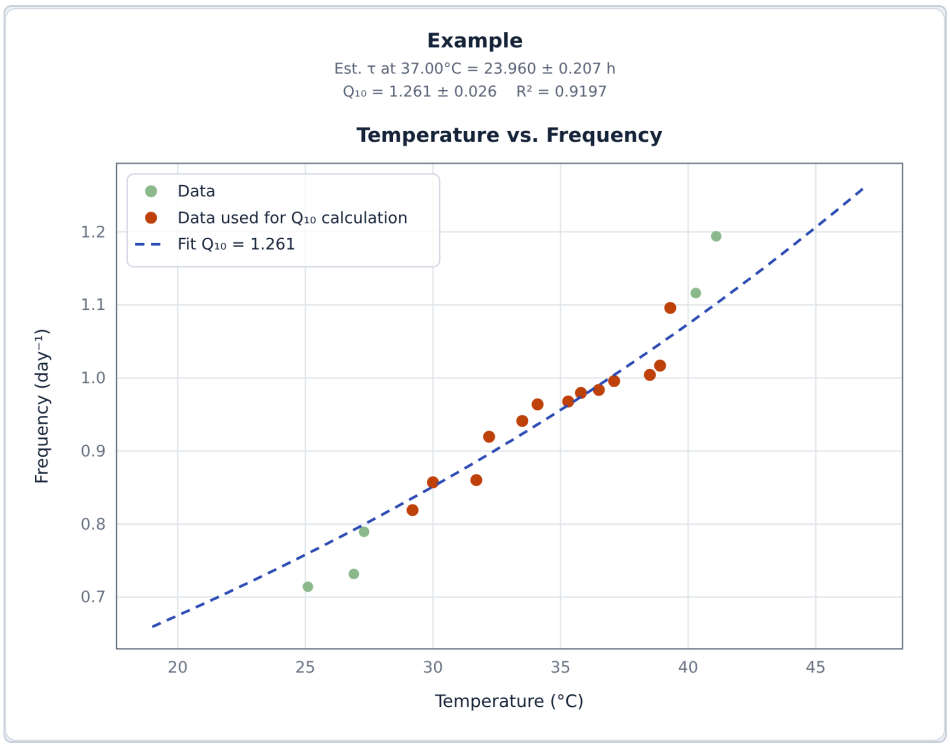


Figure 4. Period tab with a fit range of 28–40 °C. Points outside the range (light green) are plotted but not used; the legend distinguishes the two groups.

Each plot shows the title and a two-line summary (τ_b, Q_{10} and R^2) above it. The legend is placed automatically in the corner where it covers the fewest points.

5.5 The report

The **Report** tab shows the complete text results: the input table with summary statistics, the data used for the fit (when a range is set), optional tab-delimited data, and the Q_{10} and Arrhenius results. Its contents are described in Section 8; the text is identical to the file saved with **Save report (TXT)**.

5.6 Saving

Button	What is saved
Save figure (PNG)	All three plots stacked vertically under the title and summary, as a high-resolution (2×) PNG image with a white background.
Save figure (SVG)	The same figure as a vector SVG file, which can be edited in Illustrator, Inkscape or similar programs and scales without loss.
Save report (TXT)	The report as plain UTF-8 text.

File names are built from the plot title, for example `Example_q10_plots.png` and `Example_q10_results.txt` (or `q10_…` when there is no title). Files go to your browser's usual download location. The saved figure reflects the current state of the Arrhenius-line checkbox.

5.7 The "How it works" window

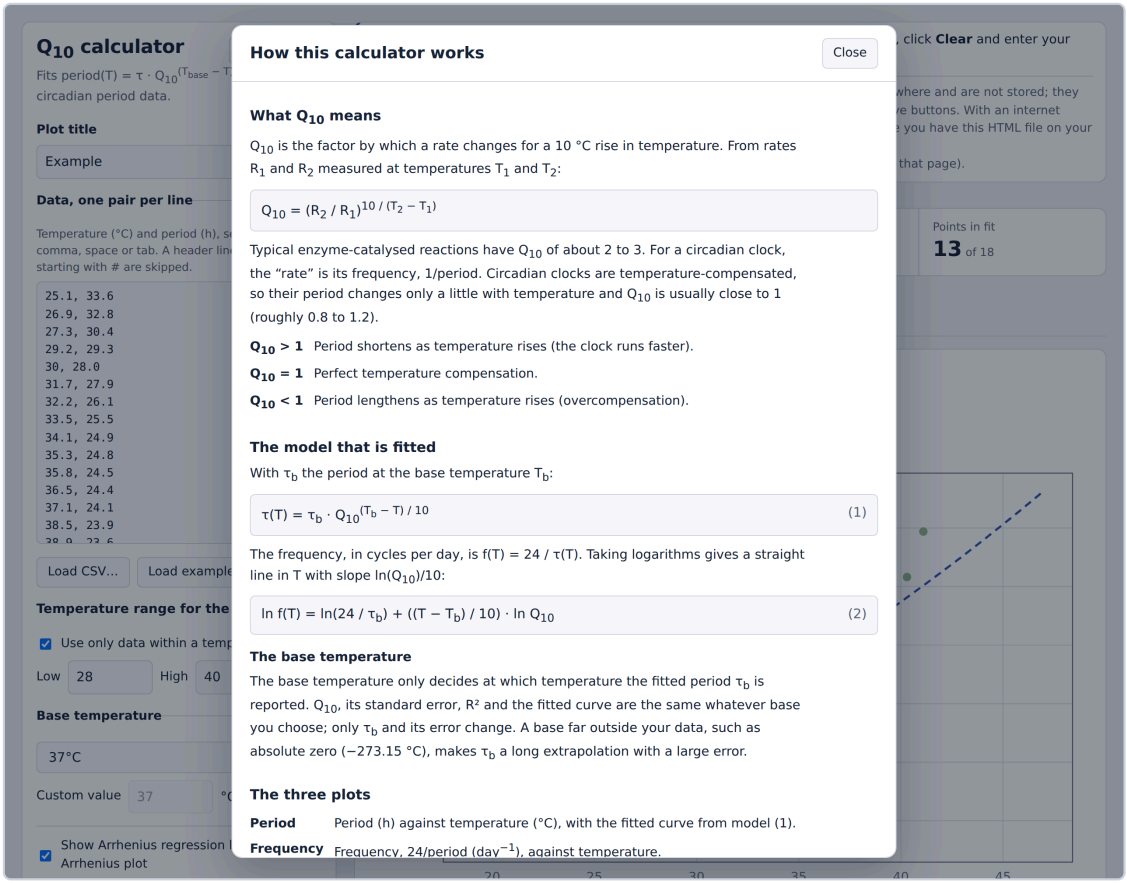


Figure 5. The "How it works" window. It closes with Close, the Esc key, or a click outside it.

The window summarises the theory, the calculation, usage and privacy (Sections 2, 3 and 12 of this manual in shorter form) and contains the download link for the HTML file.

5.8 Phones, tablets and dark mode

The page adapts to small screens: the input panel is placed above the results, the summary row wraps into two columns and plots are drawn at a size that keeps their text legible.

The page also follows the device's light or dark appearance setting. Saved figures always use a white background, whichever mode is active.

Opening a downloaded HTML file on a phone may require a file manager app or the browser's file-open function; on some devices it is easier to use the browser version on a computer.

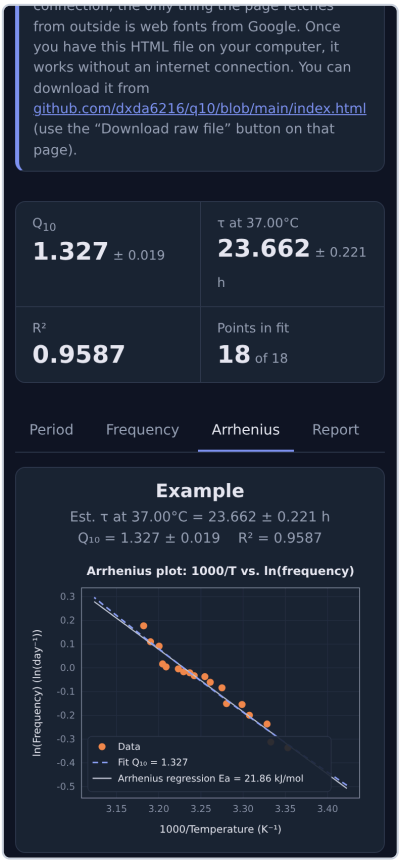


Figure 6. The browser version on a phone in dark mode.

6 Python version

6.1 Requirements

Component	Requirement
Python	3.9 or later
numpy	Any current version
SciPy	Any current version (provides `curve_fit`)
matplotlib	3.3 or later (needed for the layout of the saved three-plot figure)
Tkinter	Part of the Python standard library; usually included (see below)

All other modules used (`csv`, `math`, `re`, `statistics`, `webbrowser`) are part of the standard library. pandas is not needed.

6.2 Installation

Install the three packages with pip:

```
pip install numpy scipy matplotlib
```

Whether Tkinter is already present depends on how Python was installed:

Python installation	Tkinter
Windows or macOS, installer from python.org	Included
macOS with Homebrew Python	`brew install python-tk`
Debian, Ubuntu	`sudo apt install python3-tk`
Fedora	`sudo dnf install python3-tkinter`
Anaconda / Miniconda	Included (see Section 6.3)

To check that Tkinter works, run `python -m tkinter`; a small test window should appear.

6.3 Anaconda and Miniconda

The full Anaconda distribution already contains Python, numpy, SciPy, matplotlib and Tkinter, so nothing needs to be installed. With Miniconda, install the three packages with conda:

```
conda install numpy scipy matplotlib
```

To keep the calculator separate from other projects, you can create a dedicated environment:

```
conda create -n q10 python=3.12 numpy scipy matplotlib
conda activate q10
```

Avoid mixing `pip install` and `conda install` in the same environment, as this can break dependencies.

> **Spyder:** running the program inside Spyder can make its window freeze or fail to appear, because the Tkinter window conflicts with Spyder's own matplotlib handling. Run it from the Anaconda Prompt or a terminal instead, or set **Run → Configuration per file → Execute in an external system terminal**.
>
> **Jupyter:** the .py file is a desktop window application and is not meant to run inside a notebook; use the notebook version instead (Section 7.5).
>
> **Linux:** with older conda builds of Tk, text in the window can look jagged. This is cosmetic; `conda install -c conda-forge tk` may improve it.

6.4 Starting the program

Open a terminal (on Windows with Anaconda: the Anaconda Prompt), change to the folder containing the file and run:

```
python q10_calculator.py
```

On some systems the command is `python3`. The window opens with the example data already calculated and a notice above the results.

6.5 The window

The window has the same controls as the browser version (Figure 1), arranged slightly differently:

- **Left panel**, from top to bottom: the program name with **How it works**; **Plot title**; the data box with **Load CSV…**, **Load example** and **Clear**; the temperature range; the base temperature; the two option checkboxes; **Calculate (Ctrl+Enter)**; **Save figure…** and **Save results…**; and the status line. On a small screen the data box shrinks (and scrolls) so that the buttons stay visible.
- **Right side:** the start-up notice (demo, privacy statement and the download link for the browser version), then four tabs: **Results** (the text report) and the **Period**, **Frequency** and **Arrhenius** plots.

Each plot tab has the standard matplotlib toolbar for zooming, panning and saving that single plot. After a calculation the **Period** tab is selected.

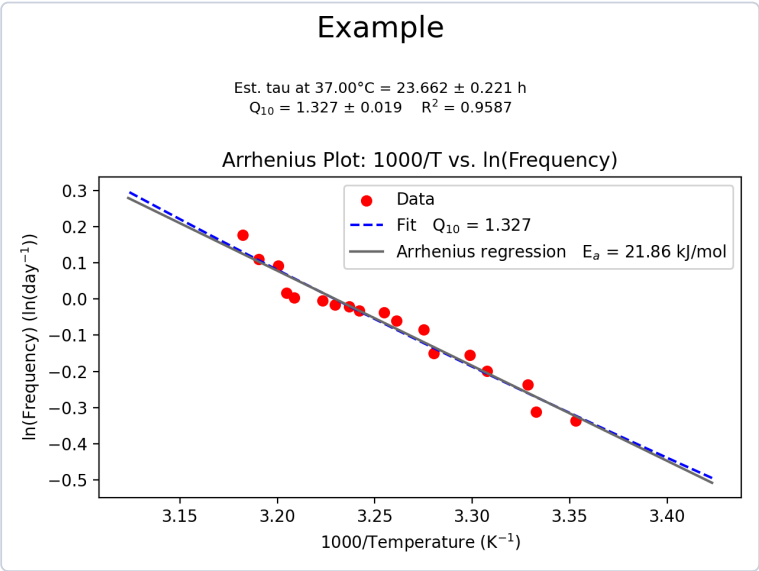


Figure 7. The Arrhenius tab of the Python version with the example data.

6.6 Step by step

The procedure is the same as for the browser version (Section 5.3): click **Clear**, enter or load your data, set the options and click **Calculate** or press Ctrl + Enter. If the input cannot be used, a dialog box titled "Cannot calculate" explains why. Switching the Arrhenius-line checkbox redraws the Arrhenius plot immediately; the tab-delimited option takes effect at the next calculation.

6.7 Saving

Save figure… asks for a file name and saves the three plots stacked in one image, in PNG (150 dpi), PDF or SVG format according to the extension you choose. PDF and SVG are vector formats suitable for publication figures.

Save results… saves the Results text as a UTF-8 text file.

A single plot can also be saved from its tab with the floppy-disk button on the matplotlib toolbar.

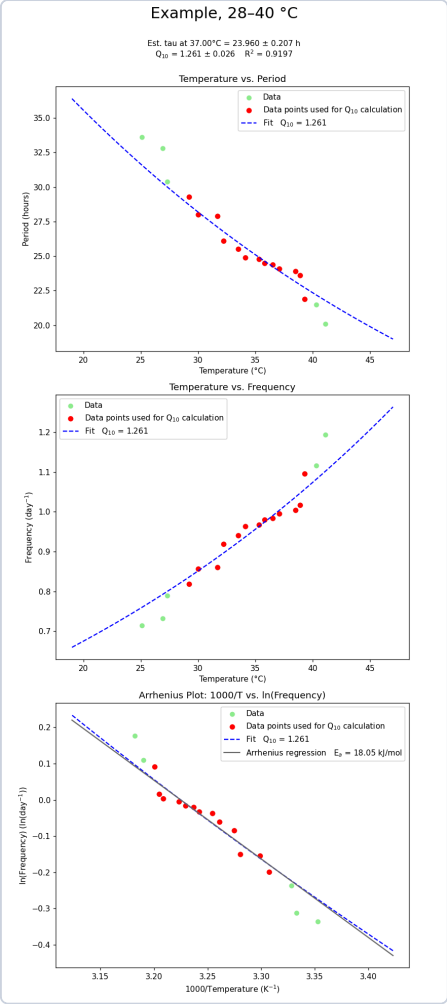


Figure 8. A figure saved by the Python version (fit range 28–40 °C).

7 Google Colab version

7.1 Opening the notebook

A Google account is required. In Google Colab (colab.research.google.com), choose **File → Open notebook**, select the **GitHub** tab, enter `dxda6216/q10` and select the notebook. Alternatively, download `circadian_period_q10.ipynb` and use **File → Upload notebook**.

The notebook has three parts: a short introduction with a privacy note, the calculator cell (shown as a form), and a "How this calculator works" section with the theory and instructions. When it opens, the form contains the example data and the output below it already shows the corresponding result.

7.2 Form fields

Field	Description
`Data_description`	Optional plot title.
`Temperatures`	Temperatures in °C, separated by commas.
`Periods`	Periods in hours, separated by commas, in the same order as the temperatures.
`Set_temperature_range`	"Yes" to fit only the points within the range set by the two sliders; "No" to use all points.
`Range_low_limit`, `Range_high_limit`	Range limits in °C (sliders, 0–65 °C, step 0.1). Used only when the range is switched on.
`Select_base_temperature`	Base temperature option (Section 4.5). Default 30 °C.
`Base_temperature`	Custom base temperature (slider, 0–100 °C), used when "Set "Base_temperature" by slider below" is selected.
`Show_Arrhenius_regression_line`	"Yes" (default) to draw the Arrhenius regression line and its E_a on the Arrhenius plot.
`Display_tab_delimited_data`	"Yes" to print the dataset, fitted curve and regression line as tab-separated columns.

7.3 Running and output

After editing the form, run the cell with **Runtime → Run all**, Ctrl + F9, or the cell's play button. Changes to the form take effect only when the cell is run again. The first run of a session may take a few seconds while Colab starts a runtime.

The output contains, in order: the demo notice (only while the form still holds the example data), the input table and statistics, the data used for the fit (when a range is set), the tab-delimited data (if chosen), the RESULTS and ARRHENIUS REGRESSION blocks, and the three plots.

If the input cannot be used, the output shows a single line starting with "Cannot calculate:" instead (Section 11).

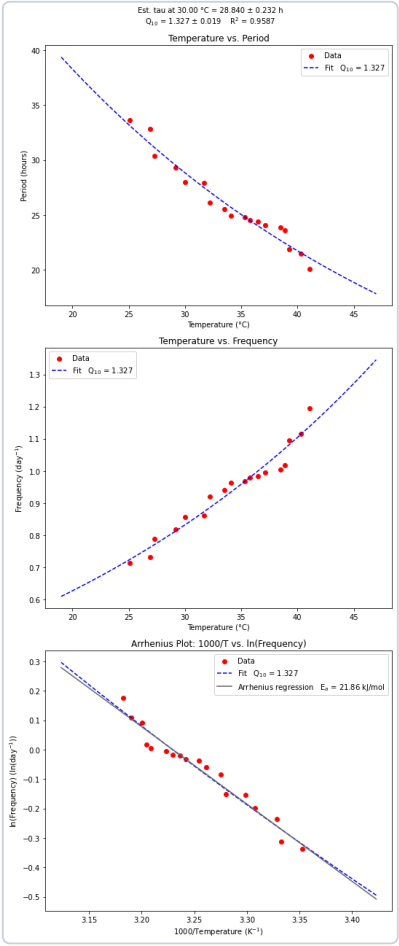


Figure 9. The plots produced by the Colab version with the example data (base temperature 30 °C).

7.4 Saving

To save the figure, right-click it and choose **Save image as…** (PNG). To keep the results, select and copy the printed text; set `Display_tab_delimited_data` to "Yes" to get columns that paste directly into a spreadsheet. Saving the notebook itself (**File → Save a copy in Drive**) also keeps the form values and the output — see Section 12 before sharing it.

7.5 Using the notebook in Jupyter

The notebook also runs in Jupyter Notebook or JupyterLab (it additionally needs pandas). The form fields are a Colab feature, so in Jupyter the code is shown in full; edit the values in the "inputs & run" section near the end of the cell (for example `Temperatures = …`) and run the cell.

8 Output reference

8.1 Results text

The text results consist of the following blocks (the Colab version prints the input tables with pandas, so their layout differs slightly):

Block	Contents
INPUT DATA	Numbered table of all temperatures and periods, followed by the minimum, maximum, average and median of each.
DATA USED FOR Q10 CALCULATION	The same for the points inside the fit range. Only shown when a range is set.
DATASET, FITTED CURVE, ARRHENIUS REGRESSION LINE	Tab-delimited columns (Section 8.3). Only shown when chosen.
RESULTS	Estimated period at the base temperature ± SE (hours); Q_{10} ± SE; R^2 of the Q_{10} fit.
ARRHENIUS REGRESSION	E_a ± SE (kJ/mol); slope d ln(f)/d(1000/T) ± SE (in units of 10^3 K); intercept ± SE; R^2 of the regression.

Example of the final two blocks (example data, base temperature 37 °C):

```
RESULTS
Estimated period length at 37.00 °C = 23.662 ± 0.221 hours
Q10 (temperature coefficient) = 1.327 ± 0.019
R² = 0.958656

ARRHENIUS REGRESSION (ln(Frequency) vs. 1000/T, data used for Q10 calculation)
Activation energy Ea = 21.859 ± 1.178 kJ/mol
Slope d ln(f) / d(1000/T) = -2.6290 ± 0.1416 (10³ K)
Intercept = 8.4911 ± 0.4612
R² = 0.955620
```

8.2 Plots

Plot	x-axis	y-axis
Temperature vs. Period	Temperature (°C)	Period (hours)
Temperature vs. Frequency	Temperature (°C)	Frequency = 24/period (day^{-1})
Arrhenius plot	1000/Temperature (K^{-1})	ln(Frequency) (ln(day^{-1}))

Element	Meaning
Red points (orange in the browser version)	Data used in the fit.
Light-green points	Data outside the chosen temperature range (plotted, not used).
Dashed blue line	The fitted Q_{10} model, labelled with its Q_{10}.

Element	Meaning
Solid grey line (Arrhenius plot only)	The Arrhenius regression line, labelled with its E_a.

In the Arrhenius plot, higher temperatures are on the *left* (smaller 1000/T) and higher frequencies at the top, so a clock that speeds up with warming gives a line falling from left to right.

8.3 Tab-delimited data

With the tab-delimited option, three tables are added, each with a header row and tab-separated columns that can be pasted directly into Excel, Google Sheets or a graphing program to redraw the plots:

- **Dataset:** Temp (°C), Period (hours) for all points;
- **Fitted curve:** Temp (°C), Period (hours) at the 200 points of the fitted curve;
- **Arrhenius regression line:** Temp (°C), 1000/T (1/K), ln(Frequency) at the same 200 temperatures.

9 Worked example

All three versions open with the following 18 data points. Running the calculator on them is a quick way to check that your copy works: you should obtain exactly the values below.

#	Temperature (°C)	Period (h)	#	Temperature (°C)	Period (h)
0	25.1	33.6	9	35.3	24.8
1	26.9	32.8	10	35.8	24.5
2	27.3	30.4	11	36.5	24.4
3	29.2	29.3	12	37.1	24.1
4	30	28.0	13	38.5	23.9
5	31.7	27.9	14	38.9	23.6
6	32.2	26.1	15	39.3	21.9
7	33.5	25.5	16	40.3	21.5
8	34.1	24.9	17	41.1	20.1

9.1 All points, base temperature 37 °C

Quantity	Value
Estimated period at 37.00 °C	23.662 ± 0.221 h
Q_{10}	1.327 ± 0.019
R^2 (Q_{10} fit)	0.958656
E_a (Arrhenius regression)	21.859 ± 1.178 kJ/mol
Slope	−2.6290 ± 0.1416 (10^3 K)
Intercept	8.4911 ± 0.4612
R^2 (Arrhenius regression)	0.955620

With a base temperature of 30 °C (the Colab default) the estimated period becomes 28.840 ± 0.232 h; all other values are unchanged.

9.2 Fit range 28–40 °C

With the range option on (Low 28, High 40), 13 of the 18 points are used. The results are Q_{10} = 1.261 ± 0.026, R^2 = 0.9197 and E_a = 18.051 ± 1.646 kJ/mol.

9.3 Reading the example

A Q_{10} of 1.33 means that in this (illustrative) dataset the clock's frequency rises by about a third for every 10 °C of warming — the period shortens from about 33.6 h at 25 °C to about 20 h at 41 °C. That is markedly less temperature-sensitive than a typical enzyme reaction ($Q_{10} \approx 2–3$), but less well compensated than many circadian clocks ($Q_{10} \approx 0.8–1.2$). Converting with equation (4) over the data range (25.1–41.1 °C) gives $E_a \approx 22.0$ kJ/mol, in close agreement with the 21.9 kJ/mol from the Arrhenius regression. Restricting the fit to 28–40 °C lowers Q_{10} to 1.26, showing that the steepest change in period lies near the ends of the range.

10 Interpreting the results

Q_{10} and its standard error

The standard error describes the statistical uncertainty of Q_{10} given the scatter of your data about the fitted curve. An approximate 95 % confidence interval is $Q_{10} \pm t \cdot SE$, where t is the 97.5 % point of Student's t distribution with n − 2 degrees of freedom: 2.57 for n = 7, 2.31 for n = 10, 2.16 for n = 15 and 2.12 for n = 18. For larger datasets t approaches 2. The standard error assumes that the residuals are independent and of similar size at all temperatures; it does not cover systematic errors such as inaccurate temperature control.

Testing temperature compensation

A Q_{10} whose confidence interval includes 1 is consistent with perfect compensation over the temperature range studied. To compare two strains or conditions, compare their Q_{10} values together with their standard errors, fitted over the same temperature range.

R^2

R^2 measures how much of the variation in period the model explains, not how well the clock is compensated. In a well-compensated clock the period barely changes, so there is little variation to explain, and R^2 can be low even when the model is adequate. Judge the fit mainly by looking at the plots: the points should scatter randomly around the curve, without a systematic bend.

Q_{10} versus E_a

Q_{10} and E_a are two descriptions of the same temperature dependence (Section 2.5). Q_{10} is the common measure in circadian biology; E_a allows comparison with biochemical reactions. If the two lines in the Arrhenius plot clearly diverge *within* the data, or the points curve away from both, neither simple model describes the data well over the whole range; consider restricting the range.

Extrapolation

The fitted curve and the regression line are drawn beyond the data for orientation only. Estimates of period far outside the measured temperatures — including τ_b at a distant base temperature — are extrapolations and should be treated with caution.

What to report

Report $Q_{10} \pm SE$, the number of points and the temperature range used, whether the points were individual measurements or means, and — if you give a period estimate — the base temperature.

11 Error messages and troubleshooting

11.1 Error messages

When the input cannot be used, no results are produced and a message explains why. The browser version shows it in the status line, the Python version in a dialog box, and the Colab version as a line starting with “Cannot calculate:”.

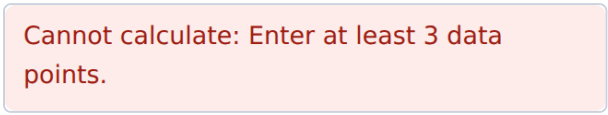


Figure 10. An error message in the status line of the browser version.

Message	Cause and remedy
Number of temperatures (*n*) and periods (*m*) differ.	A value is missing or extra. Give exactly one period per temperature.
Enter at least 3 data points.	Fewer than 3 pairs were entered. Add data.
All periods must be positive.	A period is zero or negative. Check for typos.
Range low limit must be smaller than the high limit.	Swap or correct the range limits.
Only *n* data point(s) in the selected range; need at least 3.	The range is too narrow for your data. Widen it.
Line *n*: cannot read numbers: “…” *(browser, Python)*	That line contains text that is not a number (other than one header line). Correct or delete it, or start it with # to make it a comment.
Line *n*: expected 2 values (temperature, period), found *k* *(browser, Python)*	A line has too many or too few values. Each line must hold exactly one pair.
Temperatures: could not read the numbers. Separate them with commas. *(Colab)*	The list contains something other than numbers and commas.
Enter a custom base temperature. *(browser)*	“Custom value” is selected but the field is empty.
The fit did not converge for these data. *(browser)* / Optimal parameters not found… *(Python, Colab)*	Very rarely, unusual data prevent the fit from finding a solution. Check the data for errors and outliers.
Load failed: no numeric rows found (need two numeric columns). *(browser, Python)*	The CSV file has no row whose first two cells are both numbers. Check the column order and the separator.

11.2 Troubleshooting

Problem	Solution
Browser: a save button seems to do nothing	Check the browser’s download list or permissions; some browsers ask before saving several files from one page. On iOS, files may open in a new tab, from which they can be shared or saved.

Problem	Solution
Browser: the fonts look different	Without an internet connection the page uses your system fonts. This is expected and does not affect the results.
Python: `ModuleNotFoundError: No module named 'numpy'` (or scipy, matplotlib)	Install the packages (Section 6.2) in the same Python environment that runs the program.
Python: `No module named 'tkinter'` or `_tkinter`	Install Tkinter for your system (Section 6.2).
Python: the window freezes or does not appear in Spyder	Run the program from a terminal (Section 6.3).
Python: text looks jagged on Linux with conda	Cosmetic; see Section 6.3.
Colab: changes in the form have no effect	Run the cell again after editing the form.
Colab: “Runtime disconnected”	Colab stops idle runtimes. Run the cell again; the form values are kept.

12 Privacy and data handling

Browser and Python versions

- **Calculations run on your own computer.** The fit, the plots and the report are computed locally.
- **Nothing is sent.** The data you enter and the results are never transmitted. The browser version contains no code that uploads, transmits or reports anything; the Python version makes no network connections.
- **Nothing is stored.** Data and results exist only while the page or program is open. The browser version uses no cookies or browser storage and asks the browser not to remember form contents. Files are written only when you click a Save button, and only to the place you choose.
- **Network access.** With an internet connection, the browser version downloads its typefaces from Google Fonts. That request contains none of your data, but, like any web request, lets Google see your IP address and browser type. Offline, system fonts are used and everything else works unchanged. The Python version works fully offline; matplotlib may keep its own font cache, which contains no data you enter.
- **The GitHub link.** Both versions show a link to the download page of the browser version. It is opened only if you click it.

Colab version

- **Where the calculation runs.** The code runs in a Colab runtime on Google's servers, so your data is sent to and processed by Google Colab under Google's terms.
- **What is saved.** Form values are part of the notebook's code, and the printed results and plots are kept in its output. If you save the notebook (for example to Google Drive), they are saved with it.
- **Before sharing** a notebook, remove your data from the form and choose **Edit → Clear all outputs**.
- **What the code does not do.** It makes no other network requests and writes no files.

> For unpublished or sensitive data, use the browser or Python version.

13 Differences between the versions

The calculations and results are identical. Only the interface differs:

	Browser	Python	Colab
Default base temperature	37 °C	37 °C	30 °C
Data entry	Text box or CSV file	Text box or CSV file	Two comma-separated lists
Custom base temperature	Any value	Any value	Slider, 0–100 °C
Fitting routine	Built-in Levenberg–Marquardt	SciPy `curve_fit`	SciPy `curve_fit`
Saving the figure	PNG, SVG	PNG, PDF, SVG	Right-click the image (PNG)
Saving the results	TXT file	TXT file	Copy the output
Zoom and pan	No	Yes (plot toolbar)	No
Summary row of key values	Yes	Status line	No
Explanation	“How it works” window	“How it works” window	Section in the notebook
Demo notice disappears	Clear, editing data, loading a CSV	Clear, editing data, loading a CSV	When the data differs from the example
Point colour (used in fit)	Orange	Red	Red
Dark mode, phones	Yes	No	Follows Colab

Appendix A Formula summary

Quantity	Formula
Q_{10} model	$\tau(T) = \tau_b \cdot Q_{10}^{\frac{T_b - T}{10}}$
Frequency	$f = \frac{24}{\tau}$ (day^{-1})
Log-linear form	$\ln f = \ln \frac{24}{\tau_b} + \frac{T - T_b}{10} \ln Q_{10}$
Absolute temperature	$T_K = T(°C) + 273.15$
Arrhenius regression	$\ln f = b_0 + b_1 \cdot \frac{1000}{T_K}$
Activation energy	$E_a = -1000 R b_1$, R = 8.314462618 J mol^{-1} K^{-1}
Q_{10} ↔ E_a	$E_a = \frac{R T_1 T_2 \ln Q_{10}}{10}$ (T in K)
Residual sum of squares	SSE = $\Sigma (\tau_i - \hat{\tau}_i)^2$
Standard errors	square roots of the diagonal of $s^2 (J^T J)^{-1}$, s^2 = SSE/(n − 2)
R^2	1 − SSE/SST, SST = $\Sigma (\tau_i - \bar{\tau})^2$

Appendix B Glossary

Term	Meaning
Activation energy (E_a)	The energy barrier in the Arrhenius equation; here, a measure of how steeply the clock's frequency rises with temperature, in kJ/mol.
Arrhenius plot	A plot of the logarithm of a rate against 1/T (here 1000/T); straight for a process that follows the Arrhenius equation.
Base temperature (T_b)	The temperature at which the fitted period τ_b is reported. It does not affect Q_{10}.
Frequency	Number of cycles per day, 24 divided by the period in hours.
Levenberg–Marquardt algorithm	A standard method for non-linear least squares that blends the Gauss–Newton method with steepest descent.
Period (τ)	The duration of one cycle of the rhythm, in hours.
Q_{10}	Temperature coefficient: the factor by which a rate changes for a 10 °C rise in temperature.
R^2 (coefficient of determination)	The fraction of the variation in the data explained by the fitted model.
Residual	The difference between a measured value and the value predicted by the model.
Standard error (SE)	An estimate of the statistical uncertainty of a fitted parameter.
Temperature compensation	The property of circadian clocks of keeping a nearly constant period across temperatures ($Q_{10} \approx 1$).

Appendix C Third-party components

The calculator's own code does not include code copied from other software. It relies on the following components, which are used, not redistributed, by the calculator files:

Component	Used by	Licence / terms
Python, including Tkinter	Python version	Python Software Foundation License
numpy	Python, Colab	BSD 3-Clause
SciPy	Python, Colab	BSD 3-Clause
matplotlib	Python, Colab	matplotlib License (BSD-style, PSF-based)
pandas	Colab	BSD 3-Clause
Instrument Sans, JetBrains Mono (via Google Fonts)	Browser version	SIL Open Font License 1.1; loaded from Google, not included in the file
Google Colab	Colab version	Google's terms of service